

NapCart Project Roadmap

1. Problem Definition

Small restaurants often take orders through WhatsApp and phone calls. That process creates four major problems:

1. Staff must manually collect missing customer details.
2. Order details are often incomplete or misread.
3. There is no structured order queue or reporting system.
4. Growth becomes difficult because operations depend on chat handling instead of a workflow.

This project solves that by moving order capture to a restaurant-branded website while keeping WhatsApp as the restaurant's preferred receiving channel.

2. Product Vision

Build NapCart as a reusable restaurant ordering platform where:

- Customers browse a live menu on a website.
- Customers place delivery orders through a structured checkout flow.
- Orders are saved in a central system.
- A clean order summary is automatically sent to the restaurant's WhatsApp.
- Restaurant staff manage order status through an admin dashboard.
- The platform can later support multiple restaurants with configurable branding, menus, and WhatsApp settings.

3. Business Goals

- Reduce time spent taking each order.
- Reduce mistakes caused by incomplete chat-based ordering.
- Make small restaurants look more professional.
- Create a repeatable product that can be deployed for multiple restaurant clients.
- Prepare the product for future monetization as a service or agency solution.

4. Target Users

Primary Users

- Small restaurant owners
- Restaurant managers
- Cashiers/order handlers
- Kitchen/admin staff

Secondary Users

- Customers placing food orders
- Delivery coordinators
- Future riders/drivers

5. Final System Goals

Customer Side

- Fast menu browsing
- Simple cart and checkout
- Accurate delivery information capture
- Clear order confirmation
- Optional customer order-status notifications

Restaurant Side

- Receive structured orders instantly
- View all orders in one dashboard
- Track daily sales and order statuses
- Manage menu items, categories, pricing, add-ons, and availability
- Configure WhatsApp delivery settings

Platform Side

- Reusable architecture for multiple restaurants
- Secure data separation per restaurant
- Clean admin workflows
- Upgrade-ready modules for payments, branches, promotions, and loyalty

6. Core MVP Features

Customer Website

- Restaurant landing page
- Menu categories
- Product listing with image, description, price
- Product options/variations
- Add-ons
- Cart
- Checkout
- Customer information form
- Address and delivery notes
- Cash on delivery support
- Order confirmation screen

Order Management

- Create order
- Save line items and totals
- Pending/confirmed/preparing/dispatched/delivered/cancelled statuses
- Order history
- Daily order view

WhatsApp Integration

- Send structured order notification to restaurant WhatsApp
- Retry logic for failed sends
- Delivery/read log storage

Admin Dashboard

- Login
- Overview dashboard
- Orders list
- Status updates
- Menu management
- Category management
- Product option/add-on management
- Customer list
- Basic settings
- WhatsApp settings

7. Future Features

- Online payments
- Branch support
- Multi-language menu
- Coupon and discount engine
- Minimum order rules
- Delivery zones and fees
- Rider assignment and tracking
- Inventory sync/light stock control
- Loyalty points
- Customer accounts and saved addresses
- Reorder flow
- Scheduled orders
- Analytics and exports
- Kitchen display mode
- POS integration
- CRM and marketing broadcasts
- AI-assisted upsell suggestions

8. Missing Requirements To Confirm Early

These are the main decisions that should be finalized before development:

1. Single-restaurant first or true multi-tenant from day one.
2. WhatsApp provider choice:
 - Meta WhatsApp Cloud API
 - Twilio WhatsApp
 - 360dialog

3. Whether restaurants only receive orders on WhatsApp or also update status through WhatsApp.
4. Delivery model:
 - Restaurant-managed delivery
 - Pickup only
 - Third-party rider handling later
5. Pricing model:
 - One custom install per client
 - Shared SaaS platform
 - White-label product
6. Whether customers can order as guests only, or create accounts.
7. Whether each restaurant needs a custom domain and branding panel in phase 1.
8. Geographic rules:
 - Delivery zones
 - Currency
 - Tax/VAT
 - Language
9. Whether order confirmation should be automatic or require restaurant approval.
10. Whether messages should also go to customers on WhatsApp/SMS/email.

9. Recommended Product Strategy

Best practical launch approach:

- Phase 1 product model: single codebase with multi-tenant-ready structure
- MVP market model: onboard one restaurant first
- Operational model: guest checkout, cash on delivery, delivery-only or delivery + pickup
- Messaging model: system sends order to restaurant WhatsApp, admin dashboard remains source of truth

This reduces launch risk while preserving scalability.

10. Recommended Technology Stack

Frontend

- Next.js (App Router)
- TypeScript
- Tailwind CSS
- shadcn/ui for admin components

Backend

- Next.js API routes or route handlers for early speed
- Option to split into dedicated backend service later

Database

- PostgreSQL
- Prisma ORM

Authentication

- Better Auth, Clerk, or NextAuth for admin users
- For MVP, admin-only authentication is enough

Hosting

- Vercel for frontend/app
- Supabase or Neon for PostgreSQL

File/Image Storage

- Cloudinary, Supabase Storage, or S3-compatible storage

WhatsApp

- Meta WhatsApp Cloud API preferred for direct ownership and lower vendor lock-in

Background Jobs

- Trigger.dev, Inngest, or a simple queue worker later
- For MVP, synchronous send + retry log is acceptable

Monitoring

- Sentry
- PostHog or Plausible for analytics

11. Why This Stack Fits

- Fast to build and deploy
- Type-safe end to end
- Good developer velocity for reusable SaaS-style systems
- Easy to scale from one restaurant to many
- Strong ecosystem for forms, dashboards, APIs, and integrations

12. High-Level Architecture

Core Modules

1. Storefront
2. Checkout and order engine
3. Admin dashboard
4. Menu/catalog service
5. Customer/order database
6. WhatsApp notification service
7. Settings/configuration module

Suggested Architecture Pattern

- Frontend: Next.js web app
- Backend: modular monolith
- Database: PostgreSQL
- External integration: WhatsApp API service layer

- Tenant-awareness built into schema and middleware

Why Modular Monolith First

- Easier for MVP
- Lower infrastructure complexity
- Faster iteration
- Can later split notifications, analytics, or delivery modules into services if needed

13. Multi-Tenant Approach

Even if only one restaurant launches first, design the schema for tenant isolation:

- restaurants table
- Every operational table references restaurant_id
- Domain/subdomain mapping per restaurant later
- Per-restaurant settings, branding, menu, and WhatsApp configuration

Possible routing models later:

- restaurantname.yourapp.com
- custom domain per restaurant

14. Database Structure Overview

Core Entities

Restaurants

- id
- name
- slug
- logo_url
- support_phone
- whatsapp_number
- currency
- timezone
- address
- is_active

Restaurant Settings

- restaurant_id
- order_prefix
- auto_confirm_orders
- delivery_enabled
- pickup_enabled
- default_delivery_fee
- min_order_amount
- tax_mode
- whatsapp_template_config

Admin Users

- id
- restaurant_id
- name
- email
- password_hash or auth_provider_id
- role
- is_active

Categories

- id
- restaurant_id
- name
- sort_order
- is_active

Products

- id
- restaurant_id
- category_id
- name
- slug
- description
- image_url
- base_price
- is_active
- is_featured

Product Variants

- id
- product_id
- name
- price_delta or fixed_price
- is_default

Add-on Groups

- id
- product_id
- name
- min_select
- max_select

Add-ons

- id
- addon_group_id
- name

- price

Customers

- id
- restaurant_id
- name
- phone
- email nullable
- notes
- first_order_at
- last_order_at

Customer Addresses

- id
- customer_id
- label
- full_address
- area
- latitude nullable
- longitude nullable
- delivery_notes

Orders

- id
- restaurant_id
- customer_id
- order_number
- source
- status
- subtotal
- delivery_fee
- discount_total
- tax_total
- grand_total
- payment_method
- payment_status
- fulfillment_type
- delivery_address_snapshot
- customer_phone_snapshot
- customer_name_snapshot
- notes
- placed_at
- confirmed_at
- delivered_at

Order Items

- id
- order_id
- product_id
- product_name_snapshot
- variant_name_snapshot
- unit_price
- quantity
- line_total
- item_notes

Order Item Add-ons

- id
- order_item_id
- addon_name_snapshot
- addon_price
- quantity

Order Status History

- id
- order_id
- old_status
- new_status
- changed_by
- changed_at

WhatsApp Message Logs

- id
- restaurant_id
- order_id nullable
- direction
- template_name nullable
- payload_json
- provider_message_id
- send_status
- error_message
- sent_at

Delivery Zones

- id
- restaurant_id
- name
- fee
- min_order
- is_active

15. Customer Website Flow

1. Customer lands on restaurant website.
2. Customer browses categories and products.
3. Customer opens product details.
4. Customer chooses variant and add-ons.
5. Customer adds items to cart.
6. Customer reviews cart.
7. Customer proceeds to checkout.
8. Customer enters name, phone, address, area, and notes.
9. System validates serviceability and totals.
10. Order is created in database.
11. System sends order details to restaurant WhatsApp.
12. Customer sees order confirmation page.
13. Optional: customer receives confirmation message later.

16. Order Management Flow

1. New order is created with status pending.
2. Order notification is sent to restaurant WhatsApp.
3. Order appears in dashboard instantly.
4. Staff updates order to confirmed.
5. Kitchen prepares order.
6. Order moves to preparing.
7. Dispatch happens and order moves to dispatched.
8. Delivery completes and order moves to delivered.
9. If problem occurs, order may move to cancelled.
10. All status changes are stored in history.

17. WhatsApp API Workflow

Recommended Direction for MVP

Use WhatsApp primarily as an outbound notification channel to the restaurant.

Flow

1. Customer submits order on website.
2. Backend validates payload and saves order.
3. Backend formats a clean WhatsApp message.
4. Backend sends the message via WhatsApp API provider.
5. Provider returns message ID and send status.
6. System stores API response in `whatsapp_message_logs`.
7. If sending fails, system flags error and allows retry.
8. Dashboard shows whether WhatsApp notification succeeded.

Example Message Structure

- Restaurant order number
- Customer name

- Phone number
- Address
- Delivery notes
- Ordered items
- Quantity
- Add-ons/variant details
- Subtotal
- Delivery fee
- Grand total
- Payment method

Important Product Note

For restaurant operations, WhatsApp should not be the only system of record. The dashboard/database must remain the source of truth because:

- WhatsApp delivery can fail
- Staff can miss messages
- Status updates need structure
- Reporting requires stored order data

Phase 2 WhatsApp Enhancements

- Customer confirmation message
- Order status templates to customer
- Inbound webhook processing
- Staff quick actions from WhatsApp
- Template-based multilingual messaging

18. Admin Dashboard Modules

Phase 1 Modules

- Login
- Dashboard overview
- Order list
- Order detail view
- Order status management
- Customer list
- Categories
- Products
- Variants and add-ons
- Restaurant settings
- Delivery settings
- WhatsApp settings

Phase 2 Modules

- Promo/coupons
- Branches

- Riders
- Reports and exports
- User permissions
- Content/banner management
- Payment configuration

19. Security Considerations

Application Security

- Server-side validation for all checkout inputs
- Rate limiting on order submission endpoints
- CSRF protection where relevant
- Input sanitization
- Secure auth for admin dashboard
- Role-based access control
- Session expiry and secure cookies

Data Security

- Encrypt sensitive secrets at rest
- Never expose WhatsApp API tokens to client side
- Store password hashes only if using local auth
- Minimize personal data retention
- Audit logs for admin actions

Operational Security

- Backup database regularly
- Error monitoring and alerting
- Environment variable management
- Access separation between staging and production

Compliance Considerations

- Privacy policy
- Terms of service
- Consent for customer data storage if required locally
- Data deletion approach for client requests

20. Scalability Approach

Build for These Scaling Dimensions

- More daily orders
- More concurrent customers
- More menu items
- More admin users
- More restaurants

Practical Measures

- Tenant-aware schema from day one
- Indexed database tables for orders and products
- CDN-based image delivery
- Queue or background job support for messaging
- Cached menu reads
- Separate read-heavy storefront from admin logic if needed later
- Domain mapping layer for multi-client rollout

21. Feature Priorities

P0 - Must Have for MVP

- Menu browsing
- Cart and checkout
- Order creation
- Restaurant WhatsApp notification
- Admin login
- Orders dashboard
- Order status updates
- Menu management
- Basic settings

P1 - Should Have Soon After MVP

- Delivery zones and fees
- Customer order-status notifications
- Sales reports
- Menu item availability toggles
- Basic analytics
- Retry/recovery tools for failed WhatsApp sends

P2 - Nice to Have

- Coupons
- Scheduled ordering
- Customer accounts
- Branches
- Rider management
- Loyalty
- Online payment support

22. MVP Scope Recommendation

MVP should be intentionally narrow:

- One restaurant live first
- Guest checkout only

- Cash on delivery
- Delivery and optional pickup
- Manual status updates from admin
- Outbound WhatsApp notification to restaurant only
- Basic dashboard and reporting

This is enough to prove value without overbuilding.

23. SDLC Roadmap

Stage 1: Planning

Phase 1.1 Product Discovery

- Validate the restaurant pain points with 2 to 5 real operators
- Confirm order-taking workflow
- Confirm menu complexity needs
- Confirm delivery model
- Confirm reporting expectations

Phase 1.2 Business Definition

- Define whether this is SaaS, white-label, or custom deployment
- Define onboarding process for a new restaurant
- Define pricing hypothesis
- Define support responsibility

Phase 1.3 Project Setup

- Create repository
- Define branching strategy
- Define environments: local, staging, production
- Set up issue tracker and milestones
- Create product documentation workspace

Stage 2: Requirement Analysis

Phase 2.1 Functional Requirements

- Document storefront flows
- Document checkout fields
- Document order statuses
- Document admin actions
- Document WhatsApp integration requirements
- Document menu management requirements

Phase 2.2 Non-Functional Requirements

- Performance targets
- Uptime expectations

- Security expectations
- Backup and recovery expectations
- Multi-tenant readiness expectations

Phase 2.3 Edge Case Analysis

- Out-of-stock items
- Invalid or incomplete address
- Duplicate submissions
- WhatsApp send failure
- Admin changes product price during checkout
- Restaurant temporarily closed

Stage 3: System Design

Phase 3.1 Technical Architecture

- Finalize modular monolith boundaries
- Define frontend/backend responsibilities
- Define external integrations
- Define environment configuration strategy

Phase 3.2 Data Design

- Finalize ERD
- Define table constraints
- Define indexes
- Define status enums
- Define audit strategy

Phase 3.3 UX/UI Design

- Design storefront pages
- Design mobile-first menu experience
- Design checkout UX
- Design admin dashboard
- Design order-detail workflow

Phase 3.4 API Design

- Define checkout API
- Define admin order APIs
- Define menu CRUD APIs
- Define settings APIs
- Define WhatsApp service contracts

Stage 4: Development

Phase 4.1 Foundation Setup

- Initialize Next.js project

- Add TypeScript, linting, formatting
- Configure Tailwind and UI primitives
- Configure Prisma and PostgreSQL
- Configure env management
- Configure auth

Phase 4.2 Database and Core Models

- Create schema
- Create migrations
- Seed example restaurant, categories, and products
- Add indexes and constraints

Phase 4.3 Storefront

- Build homepage/menu page
- Build category filtering
- Build product detail modal/page
- Build cart state
- Build checkout form
- Build confirmation page

Phase 4.4 Order Engine

- Validate checkout payload
- Calculate totals server-side
- Create order records
- Store snapshot fields
- Prevent duplicate submissions

Phase 4.5 WhatsApp Integration

- Build provider adapter layer
- Build order message formatter
- Send order notification
- Store send logs
- Add retry handling

Phase 4.6 Admin Dashboard

- Build auth-protected admin area
- Build dashboard summary cards
- Build order list and filters
- Build order detail page
- Build status update actions
- Build category/product management
- Build settings pages

Phase 4.7 Operational Hardening

- Add rate limits
- Add audit logs

- Add error boundaries and logging
- Add monitoring hooks

Stage 5: Testing

Phase 5.1 Unit Testing

- Pricing calculations
- Cart/order validation
- Delivery fee logic
- Message formatter logic
- Status transition logic

Phase 5.2 Integration Testing

- Checkout to order creation
- Order creation to WhatsApp send
- Admin status updates
- Product CRUD flows

Phase 5.3 Manual QA

- Mobile storefront testing
- Slow network testing
- Invalid input testing
- Restaurant closed scenario
- WhatsApp failure scenario

Phase 5.4 UAT

- Test with one real restaurant
- Collect staff workflow feedback
- Measure order handling time improvement

Stage 6: Deployment

Phase 6.1 Infrastructure

- Create Vercel project
- Provision database
- Configure storage
- Configure secrets
- Set up staging and production

Phase 6.2 Launch Prep

- Seed production menu
- Set up restaurant branding
- Configure WhatsApp credentials
- Configure domain
- Create backup policy

Phase 6.3 Go Live

- Smoke test live flows
- Place test orders
- Verify message delivery
- Monitor errors closely for first week

Stage 7: Maintenance and Scaling

Phase 7.1 Post-Launch Support

- Fix launch bugs
- Improve UX friction points
- Add operational reports
- Review error logs weekly

Phase 7.2 Product Expansion

- Add payments
- Add delivery zones
- Add customer notifications
- Add branches
- Add riders

Phase 7.3 Platform Expansion

- Add tenant onboarding tools
- Add white-label theming
- Add subscription billing for restaurant clients
- Add central super-admin panel

24. Step-by-Step Task Sequence

Sprint 0: Strategy and Specification

- Finalize MVP scope
- Select WhatsApp provider
- Confirm single-restaurant launch strategy
- Write PRD
- Write ERD
- Write user flows
- Create UI wireframes

Sprint 1: Foundation

- Initialize repo
- Set up app shell
- Configure database and ORM
- Create core schema
- Add seed data

- Build auth shell

Sprint 2: Storefront MVP

- Menu listing
- Product details
- Cart
- Checkout form
- Validation
- Confirmation page

Sprint 3: Order Pipeline

- Order creation backend
- Status model
- Totals engine
- Order persistence
- WhatsApp notification
- Failure handling

Sprint 4: Admin MVP

- Dashboard overview
- Orders table
- Order detail page
- Status actions
- Product/category CRUD
- Settings pages

Sprint 5: QA and Launch

- Tests
- Bug fixing
- UAT with restaurant
- Deployment
- Launch support

25. Suggested Timeline

For one strong developer or a small team:

- Discovery and specification: 1 to 2 weeks
- Design and architecture: 1 week
- MVP development: 4 to 6 weeks
- Testing and launch prep: 1 to 2 weeks

Estimated MVP total:

- 7 to 11 weeks depending on design polish, WhatsApp approval speed, and feedback cycles

26. Main Risks

1. WhatsApp API onboarding or template approvals may slow launch.

2. Restaurants may want custom operational flows that break standardization.
3. Delivery area logic can become complex quickly.
4. Menu modeling can get messy if variants and add-ons are not designed well.
5. If WhatsApp becomes the only operational view, reporting and reliability will suffer.

27. Key Recommendations

1. Build the dashboard as the true operational system, not WhatsApp alone.
2. Keep MVP limited to guest checkout and cash on delivery.
3. Design database and routing as multi-tenant-ready, even if only one restaurant launches first.
4. Use modular monolith architecture first.
5. Abstract the WhatsApp provider behind a service layer so you can change vendors later.
6. Launch with one real restaurant, then generalize after observing real behavior.

28. Immediate Next Deliverables

After this roadmap, the next best execution order is:

1. Finalize MVP feature list and assumptions.
2. Choose the exact tech stack and WhatsApp provider.
3. Produce a PRD.
4. Produce the database ERD.
5. Produce wireframes for storefront and admin dashboard.
6. Set up repository and start Sprint 1.